Your Paper

You

April 16, 2026

Abstract

Your abstract.

1 Lecture review

1.1 Relative Pose Estimation of the Robot

1.1.1 Coordinate Systems and the Chain Rule

To estimate the relative position of the robot, multiple reference frames are introduced. In this work, the fixed world reference frame is denoted as the global coordinate system $\{W\}$, while the coordinate system attached to the robot body is denoted as the local coordinate system $\{B\}$. The global coordinate system provides a unified reference for describing the robot pose in the environment, whereas the local coordinate system is more suitable for representing short-term motion increments.

The pose of the robot in the global coordinate system can be written as the homogeneous transformation matrix

$$^{W}T_B = \begin{bmatrix} ^{W}R_B & ^{W}p_B \\ 0 & 1 \end{bmatrix}$$

where $^{W}R_B$ is the rotation matrix of the robot body frame relative to the global frame, and $^{W}p_B$ is the position vector of the robot origin in the global frame. If a point P is represented in the local coordinate system as ^{B}p, then its coordinates in the global coordinate system can be expressed as

$$^{W}p = {}^{W}R_B \, {}^{B}p + {}^{W}p_B.$$

This equation shows that a point is first rotated and then translated when transformed from the local frame to the global frame.

To update the pose across consecutive time steps, the chain rule of coordinate transformation is adopted. Let the robot body frames at time $k-1$ and k be denoted by $\{B_{k-1}\}$ and $\{B_k\}$, respectively. Their relationship can be written as

$$^{W}T_{B_k} = {}^{W}T_{B_{k-1}} \, {}^{B_{k-1}}T_{B_k}.$$

Here, $^{B_{k-1}}T_{B_k}$ represents the local pose increment between two adjacent time steps. Based on this chained relationship, the robot pose can be recursively updated in the global frame, or equivalently represented relative to any chosen reference frame.

1.1.2 Coordinate Transformation Update Based on Velocity and Attitude

When the local displacement increment and the previous attitude of the robot are known, the current coordinate transformation can be updated recursively. Let Δp_{k-1} denote the displacement increment in the local frame at time $k-1$. The robot position in the global coordinate system at time k can then be computed as

$$p_k = p_{k-1} + R_{k-1}\Delta p_{k-1},$$

where p_{k-1} is the global position vector at the previous time step, and R_{k-1} is the rotation matrix of the robot body frame relative to the global frame. This equation indicates that the local motion increment must first be rotated into the global frame before being added to the previous position.

The local displacement increment can be approximated from the robot velocity and the sampling interval. If the forward linear velocity is v and the sampling interval is Δt, then

$$\Delta p_{k-1} = \begin{bmatrix} v\Delta t \\ 0 \\ 0 \end{bmatrix}.$$

This formulation assumes that the robot motion within one time step mainly occurs along the forward axis of the body frame. In practice, the robot heading may change continuously during motion. Therefore, a smaller Δt usually leads to better consistency between the assumed motion direction and the actual attitude, which improves the accuracy of the estimated position.

Using this recursive formulation together with the chain rule, the robot pose can be expressed not only in the global frame but also relative to any selected reference frame. However, since this method depends on the commanded velocity, the sampling interval, and the attitude measurement result, the estimation error accumulates over time. Therefore, external position feedback or other correction information should be introduced periodically to suppress long-term drift.

1.2 Theoretical Background of 2D Map Generation

1.2.1 Two-Dimensional Grid Map and Its Role

To use a 3D point cloud for subsequent route search and path planning, the environment is first discretized into a two-dimensional grid map. The space is divided into regular cells with a fixed resolution, and each cell represents a local region of the environment. In this way, the originally continuous and large-scale point cloud is compressed into a regular discrete representation.

The main role of the 2D grid map is to provide a discretized search space for path-planning algorithms. Direct search on the raw point cloud is computationally expensive because the data scale is large and the adjacency relationship among points is complicated. After grid discretization, path search only needs to be performed among a finite number of cells, which makes the neighborhood relationship clearer and reduces the computational burden.

In addition, the 2D grid map helps control the growth of complexity as the map becomes larger. Local geometric information in each region can be summarized by a traversal cost *cost*, where a larger value indicates more difficult traversal and a smaller value indicates better traversability. In this way, the environment is represented as a collection of cost-associated cells, which provides a compact and efficient input for subsequent path planning.

1.2.2 Ground Layer Selection

During the construction of a two-dimensional grid map, a single grid cell may contain more than one layer of points. In addition to the actual ground, points may also come from the bottom of obstacles, suspended structures, or noise. Therefore, the lowest point in a cell cannot be directly regarded as the ground. To address this issue, candidate layers are first constructed along the vertical direction in each grid cell, and then the layer that most likely represents the actual ground is selected.

Let the candidate layer set of the i-th grid cell be

$$\mathcal{L}_i = \{L_{i,1}, L_{i,2}, \ldots, L_{i,m_i}\}$$

where m_i is the number of candidate layers in the cell. For each candidate layer $L_{i,k}$, its representative height is defined as

$$h_{i,k} = \frac{1}{N_{i,k}} \sum_{p \in L_{i,k}} y(p)$$

where $N_{i,k}$ is the number of points in $L_{i,k}$, and $y(p)$ is the height coordinate of point p.

Since the actual ground is usually locally continuous, ground-layer selection should depend not only on the height distribution inside the current cell but also on the ground height of neighboring cells. Let $\hat{h}_{\mathcal{N}(i)}$ denote the reference ground height in the neighborhood of the i-th cell. Then the ground layer of the current cell can be defined as

$$L_i^* = \arg\min_{L_{i,k} \in \mathcal{L}_i} \left| h_{i,k} - \hat{h}_{\mathcal{N}(i)} \right|$$

which means that the candidate layer whose height is closest to the neighborhood reference ground height is selected as the ground layer. To avoid incorrectly classifying a layer with a large height difference as ground, an additional continuity constraint can be introduced:

$$\left| h(L_i^*) - \hat{h}_{\mathcal{N}(i)} \right| < \tau_h$$

where τ_h is the ground-continuity threshold. This candidate-layer selection strategy based on neighborhood continuity can extract the actual ground more robustly and reduce the interference caused by suspended structures, obstacle boundaries, and local noise.

1.2.3 Eigenvalues and Eigenvectors

After ground-layer selection, the local geometric characteristics of each grid cell still need to be analyzed in order to evaluate traversability. In this work, eigen-analysis is not performed on the entire map as a whole. Instead, it is carried out separately for each grid cell by constructing a covariance matrix from its local point set. As a result, the obtained eigenvalues and eigenvectors describe the local surface properties of each individual cell rather than the global properties of the entire map.

For the i-th grid cell, let its local point set be

$$\mathcal{P}_i = \{p_1, p_2, \ldots, p_{N_i}\}, \quad p_k \in \mathbb{R}^3$$

and let its centroid be

$$\bar{p}_i = \frac{1}{N_i} \sum_{k=1}^{N_i} p_k$$

Then the covariance matrix of this cell is defined as

$$C_i = \frac{1}{N_i} \sum_{k=1}^{N_i} (p_k - \bar{p}_i)(p_k - \bar{p}_i)^T$$

The eigen-decomposition of C_i is written as

$$C_i n_{i,j} = \lambda_{i,j} n_{i,j}, \quad j = 1, 2, 3$$

with

$$\lambda_{i,1} \leq \lambda_{i,2} \leq \lambda_{i,3}$$

If the local point set in the cell is approximately distributed on a plane, then the eigenvector corresponding to the smallest eigenvalue, $n_{i,1}$, can be regarded as the approximate normal vector of the local ground surface. Let the unit vertical direction be

$$v = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

This eigenvector can also be treated as the surface orientation vector for slope evaluation. Then the local slope of the cell can be expressed as

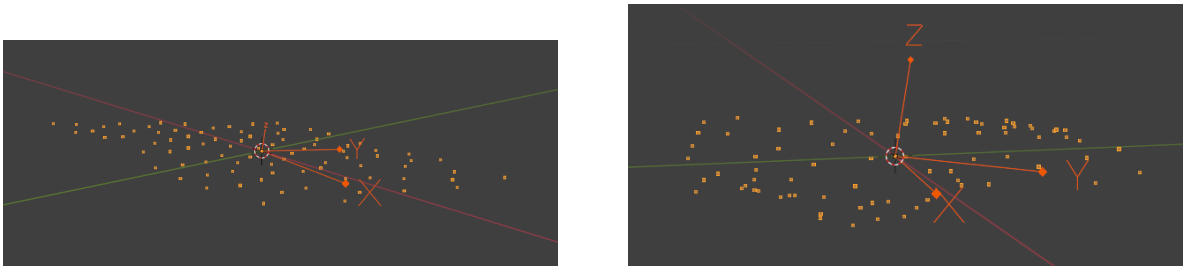
$$\theta_i = \arccos\left(\frac{n_{i,1} \cdot v}{\|n_{i,1}\| \, \|v\|} \right)$$

where a larger θ_i indicates a larger local surface inclination.

At the same time, the smallest eigenvalue $\lambda_{i,1}$ reflects the dispersion of points along the normal direction and can therefore be used to characterize the local roughness or unevenness of the cell. A normalized roughness index can be defined as

$$r_i = \frac{\lambda_{i,1}}{\lambda_{i,1} + \lambda_{i,2} + \lambda_{i,3}}$$

where a larger r_i indicates a rougher or less stable local surface.



(a) Cell with smaller minimum eigenvalue.

(b) Cell with larger minimum eigenvalue.

Figure 1: cells with different minimum eigenvalues.

Therefore, the eigenvector associated with the smallest eigenvalue mainly describes the local directional property of each grid cell, while the smallest eigenvalue itself reflects the stability of the cell along the surface normal direction. By analyzing slope and roughness on a cell-by-cell basis, the local traversability of different regions can be quantified and used for subsequent cost-map construction. As illustrated in Fig. 1, two cells may exhibit similar slope angles while still having different roughness levels. The annotated z-vector in each figure corresponds to the eigenvector associated with the smallest eigenvalue, and the angle between this vector and the world z-axis represents the local slope, which is approximately 17° in both cases. Compared with Fig. 1a, the cell in Fig. 1b contains a local pit and is therefore geometrically more irregular, leading to a larger minimum eigenvalue.

2 Methods

2.1 Swarm Scanning and Centralized Processing Architecture

This work adopts a swarm-based scanning strategy, in which multiple robots scan the environment in parallel. Compared with a single-robot solution that scans the scene region by region, this approach can collect environmental information over a larger area within a shorter time. Meanwhile, the number of deployed robots can be adjusted according to the environment size and practical constraints such as time and cost, which makes the overall scheme more flexible.

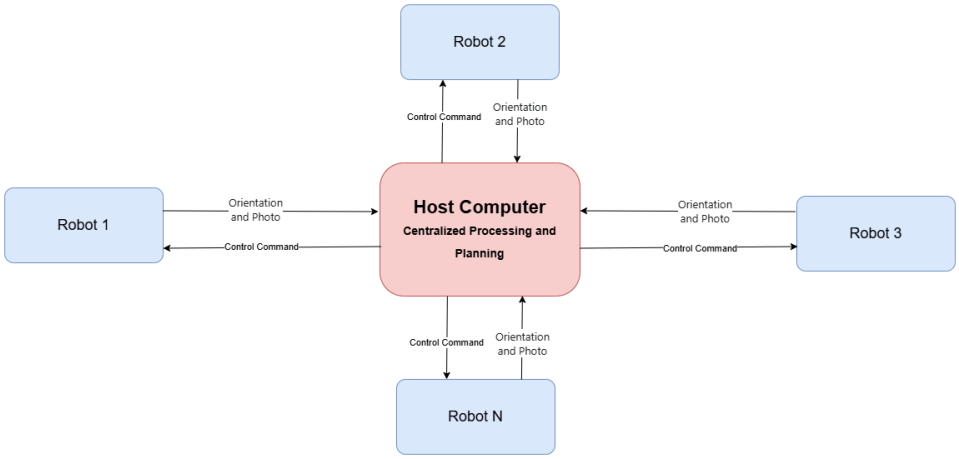


Figure 2: Centralized processing architecture for swarm monocular SLAM robots.

Since AI-based models are used in the subsequent processing stages, the system requires relatively high computing power. However, if every individual SLAM robot were equipped with a high-performance NPU or other dedicated AI hardware, the total hardware cost in a multi-robot scenario would increase significantly, as shown in Fig. 2. To reduce this cost, a centralized data-processing architecture is adopted. Under this architecture, all SLAM robots are treated as executors and sensors. Their onboard functions are limited to motion, image acquisition, and pose-related data collection,

while computationally intensive AI inference is not performed locally. Instead, all collected data are transmitted through the network to a high-performance host computer, which is responsible for unified data processing, map construction, and path planning. In this way, the high computational load is concentrated on a single terminal, effectively reducing the hardware cost of each robot.

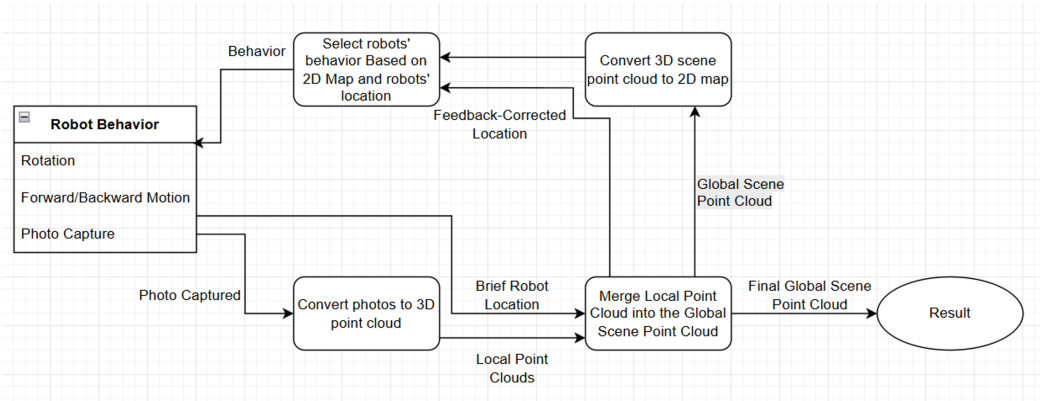


Figure 3: Overview flow chart of single-robot control and data acquisition.

The overall operation flow is shown in Fig. 3. After the host computer receives the images and coarse pose estimates uploaded by the robots, it first applies Depth Pro to convert the images into depth maps, and then reconstructs the point cloud corresponding to each image. Based on the coarse pose estimates, the host computer performs post-processing on each point cloud and merges it into the global point cloud. At the same time, the current robot pose can be further refined using the feedback from the merging error, so that a more accurate pose estimate is obtained. On this basis, the global point cloud is further converted into a 2D map, which is used to determine traversable regions, blocked regions, unexplored regions, and the traversal cost of different areas. The path-planning module then generates paths and motion commands for each robot according to the refined poses and the current 2D map. When all robots have completed the scanning task, that is, when no further valid path can be planned, the scanning process ends. Finally, the merged global point cloud is exported as the output of the complete scanning task.

2.2 RTOS-Based Robot Control Architecture

To improve the real-time performance and structural clarity of the onboard control software, the robot-side system was implemented using an RTOS-based architecture. The whole system contains four core tasks: the WiFi feedback task, the robot control task, the photo-reading task, and the position-estimation task. These tasks cooperate with different responsibilities so that communication, motion execution, image update, and pose estimation can proceed in parallel.

The software maintains a unified global state object, denoted as `robot state`, which stores the key runtime status of the robot, including the movement mode, motor PWM, actual moving speed, latest captured image, current camera parameters, current IMU output, error logs, reference coordinate frame, and current relative pose. This state object also serves as the interface between the communication layer and the execution layer. As illustrated in Fig. 4, the left side of the architecture can be regarded as the communication layer, while the right side can be regarded as the sensing and execution layer. The communication layer does not directly read or write hardware. Instead, it only reads and modifies the shared `robot state` object according to the received packets. Conversely, the execution layer does not directly communicate with the communication layer. It continuously updates `robot state` from hardware observations and maps the relevant state attributes to hardware actions. In this way, the two parts are decoupled through the shared state object rather than directly invoking each other, which helps avoid blocking and makes the program structure clearer.

Among the four tasks, the WiFi feedback task has the highest priority. It is responsible only for communication, as well as reading and modifying the shared state object, but it does not directly perform time-consuming operations such as IMU reading, image acquisition, or pose computation. This task remains suspended during idle periods and is awakened only when a communication packet is received. The robot control task has the second-highest priority and is also suspended most of

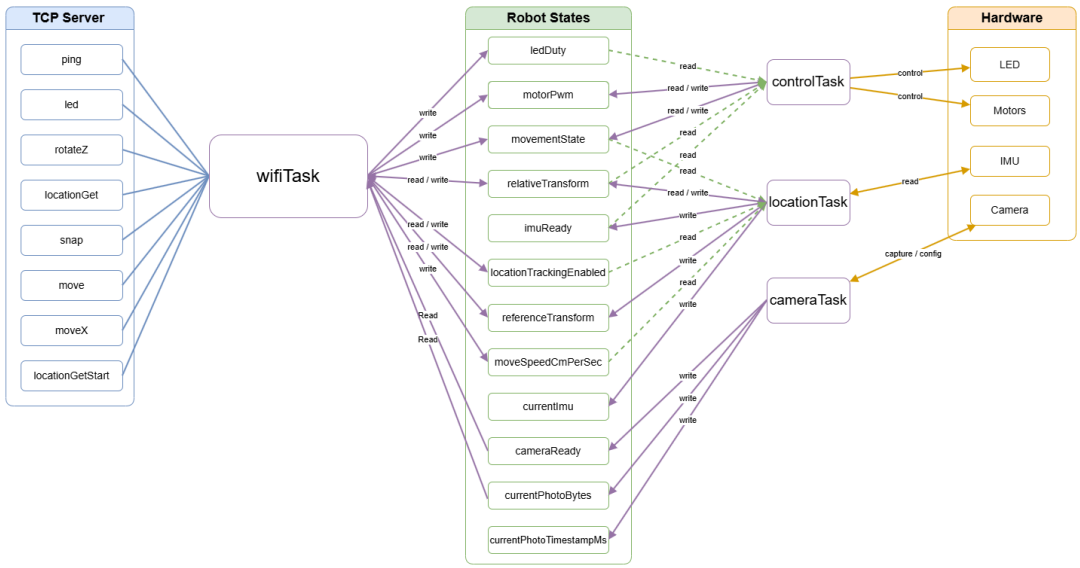


Figure 4: RTOS-based robot control architecture centered on the shared `robot state` object.

the time. It is awakened after the WiFi feedback task updates motion-related parameters, and then updates the robot motion according to the movement mode and PWM stored in the shared state.

The photo-reading task and the position-estimation task have the same lower priority and run periodically in the background. The photo-reading task polls the camera at a fixed interval and continuously refreshes the latest image buffer. The position-estimation task reads the IMU periodically and updates the relative pose. By running these two tasks continuously in the background, the system avoids blocking caused by temporary hardware access when a command arrives.

To guarantee consistency in multi-task access to shared data, all shared states are protected by a mutex. In addition, any shared result must be written back to the state object only after the complete computation is finished, rather than through staged partial updates. This ensures that the WiFi feedback task always reads complete and internally consistent data when responding to external requests.

2.3 2D Map Generation and Cost Modeling

To convert the fused 3D point cloud into a representation suitable for path planning, the environment is discretized into regular 2D grid cells. Each cell can only be assigned one of three states: `block`, `costing`, or `unknown`. Here, `block` means that the region is not traversable, `costing` means that the region is traversable but associated with a traversal cost, and `unknown` means that the region has not yet been scanned or has not been reliably observed. The robot is allowed to move only through cells labeled as `costing`. For `costing` cells, the traversal cost ranges from 0 to 255, where a larger value indicates worse road condition and greater traversal difficulty. If the final cost of a cell is greater than or equal to 255, the cell is also treated as `block`.

Each grid cell is defined as 5 cm $\times$ 5 cm. This resolution is selected according to the robot size and a safety margin. The robot body is approximately $10 \times 10 \times 11$ cm, and the diameter of its self-rotation coverage is about

$$2\sqrt{5^2 + 5^2} \approx 14 \text{ cm}.$$

To reserve additional redundancy, the robot is approximated as a $15 \times 15 \times 15$ cm cube. Under this setting, the 5 cm $\times$ 5 cm grid provides a convenient resolution for local traversability evaluation. If the neighboring cells covered by the projected robot footprint are all labeled as `costing`, the region can be considered traversable.

Costing computation. The 2D map is generated by first performing ground-layer selection on the point cloud. After the ground layer is determined for each cell, the system checks whether an obstacle

exists within a height threshold H_{th} above the ground layer. If such an obstacle is detected, the cell is directly labeled as `block`. In this work, the height threshold is set to

$$H_{th} = 15 \text{ cm},$$

which is slightly larger than the robot height of about 11 cm in order to provide a safety margin.

If no obstacle is found within the threshold, the cell enters the `costing` stage. The traversal cost is computed from both slope and local complexity. The slope is estimated from the local eigenvector. Since the robot can reliably climb slopes up to about 30°, the interval from 0° to 30° is linearly mapped to a cost range of 0 to 255. If the slope exceeds 30°, the corresponding cell is directly labeled as `block`.

In addition, the local complexity is computed from the normalized eigenvalue-based roughness measure. A larger normalized value indicates a more irregular surface. When the three eigenvalues are equal, the local point set can be regarded as being uniformly distributed in a 5 cm × 5 cm × 5 cm volume, which corresponds to the maximum complexity. Its normalized value is approximately

$$\sqrt{\frac{1}{3}} \approx 0.6.$$

When the complexity is one tenth of this maximum value, the normalized result becomes

$$\sqrt{\frac{1}{30}} \approx 0.18.$$

Therefore, the complexity threshold is set to 0.18. This threshold is chosen because the robot can stably pass obstacles of about 0.5 cm without significantly affecting pose estimation, so one tenth of the maximum complexity is taken as a practical tolerance level.

The relationship between complexity and actual obstacle severity is not linear, but closer to a quadratic trend. Therefore, the complexity cost is defined as

$$cost(c) = 255 \left(\frac{c}{0.18}\right)^2,$$

where c is the local complexity value.

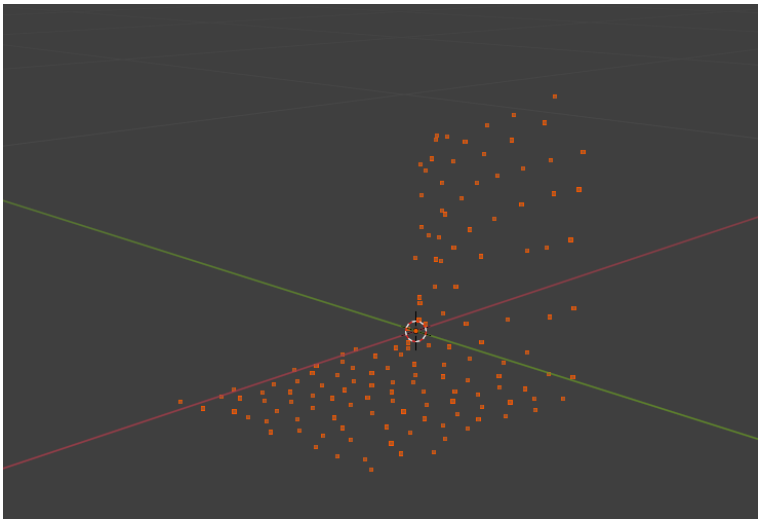
Environment types and state conversion. Four typical environment types are considered in this work: walls or steps, suspended obstacles, planar or sloped regions, and pits or cliff-like regions.

Walls, steps, and suspended obstacles are handled in a similar manner. After ground-layer selection, these structures usually appear as elevated obstacles within a limited height range above the detected ground layer. Therefore, once such an obstacle is found within H_{th}, the corresponding cell is directly classified as `block`. Planar regions and slopes are mainly represented through slope and complexity, and are therefore converted into different `costing` values. Small local obstacles are also reflected by increased local complexity, which raises the traversal cost of the corresponding cells.

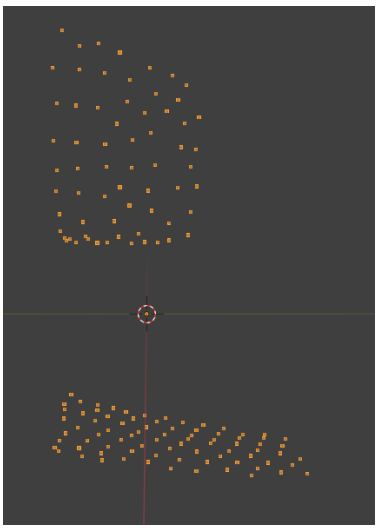
Pits and cliffs exhibit a different behavior. Due to viewpoint limitations, these regions are often difficult to observe completely during early scanning stages, and their edges or walls may not be reconstructed reliably. As a result, such regions are usually labeled as `unknown` first, which prevents the robot from entering potentially dangerous areas. After more observations are collected, these regions may later be reclassified as `costing` or `block`. Typical examples of these environment types are shown in Fig. 5.

Map overwrite and merging. In the global map construction process, each grid cell is associated with its corresponding real-world coordinate. To save computational cost, each frame of map merging updates only the changed point-cloud data, and the 2D map module performs overwrite-based merging on the corresponding cells. This strategy is based on the assumption that, under ideal conditions, repeated point-cloud merging gradually reduces error and produces more accurate geometry. Therefore, newly observed results are used to overwrite previous ones. Through this mechanism, cells originally marked as `unknown` can also be updated into `costing` or `block`, allowing the global 2D map to evolve continuously during the scanning process.

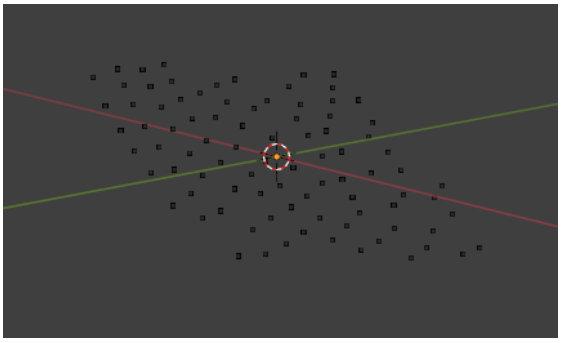
References



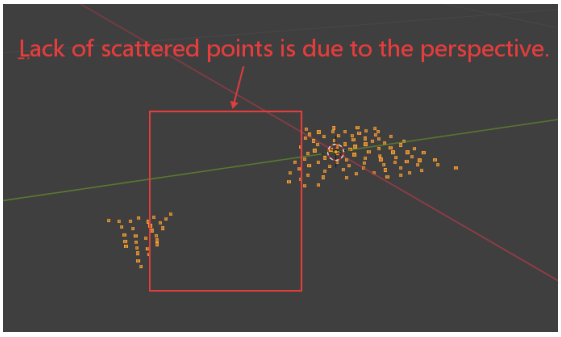
(a) Wall or step obstacle.



(b) Suspended obstacle.



(c) Slope or traversable uneven region.



(d) Pit or cliff-like region.

Figure 5: Typical obstacle and terrain types considered in the 2D map generation process.